

Practical Lesson 2

Axes and Design

MECD 2024/25

Visualização Avançada de Dados

Evgheni Polisciuc

Agenda

- 1. 2D Cartesian axis**
2. Polar axis
3. Color “axis”
4. Title and Legends

2D Cartesian axis

- There are two axes in plotly: **xaxis** and **yaxis**
- These are the part of **go.layout object** (**go.layout.XAxis** and **go.layout.YAxis**)
- There are other kinds of axis in plotly (e.g., polar axis, geo axis), which will be covered in other classes.
- There are different types of axis: '**linear**', '**log**', '**date**', '**category**', '**multicategory**' – if not specified, the type is auto-detected
- Ex. **fig.update_xaxes(type='category')**

2D Cartesian axis

- Different groups of properties:
 - **type** of the axis.
 - **title** of the axis (text, font)
 - Tick values (location of tick markers) and tick labels. Note that the tick labels and grid lines are placed at tick values
 - **lines**: grid lines, axis lines, zero line
 - **range** of the axis
 - **domain** of the axis

Title

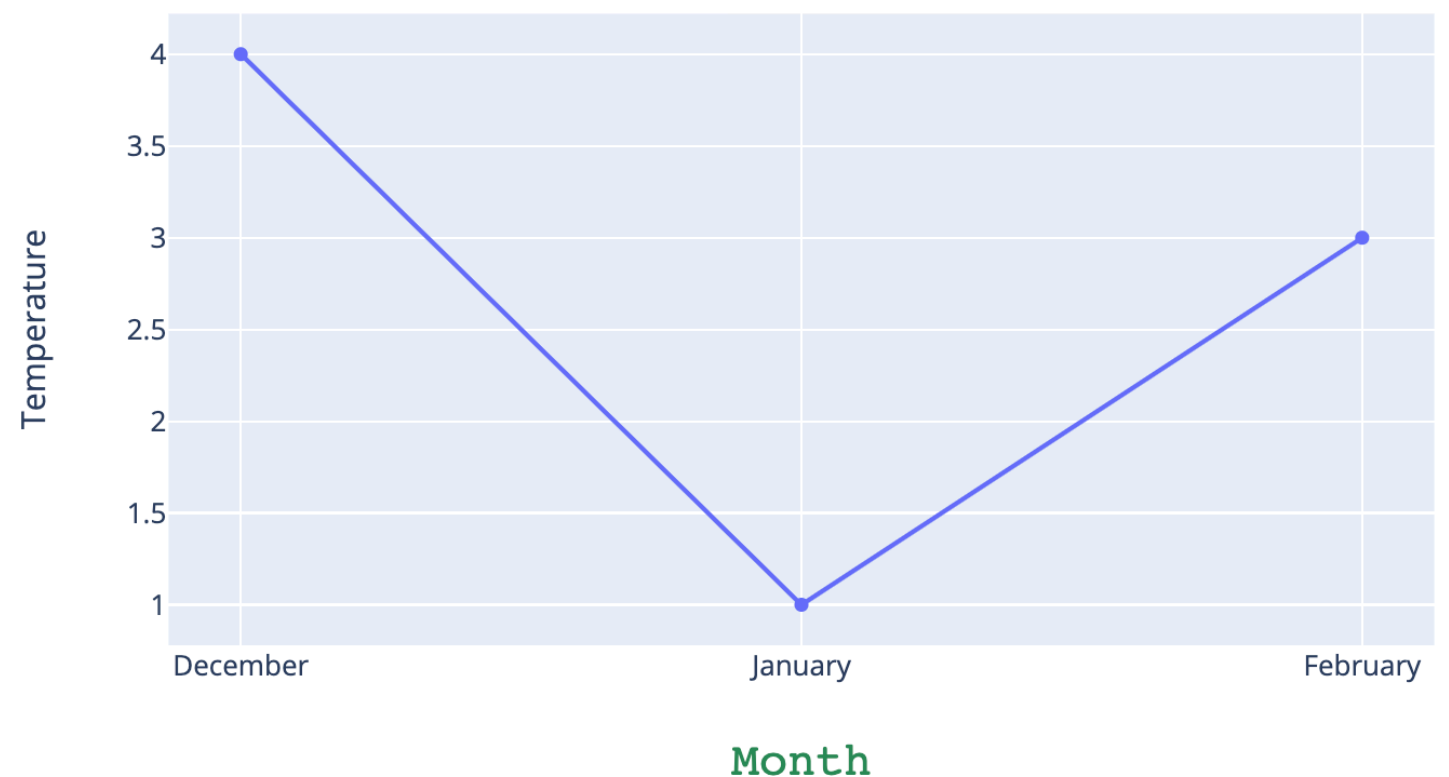
Text, font, and position

```
fig = go.Figure(go.Scatter(  
    mode = "lines+markers",  
    y = [4, 1, 3],  
    x = ["December", "January", "February"]))
```

```
fig.update_xaxes(  
    title_text = "Month",  
    title_font = dict(size=20, family='Courier', color='seagreen'),  
    title_standoff = 25)
```

```
fig.update_yaxes(  
    title_text = "Temperature",  
    title_standoff = 25)
```

standoff: Sets the standoff distance (in px) between the axis labels and the title text



Ticks

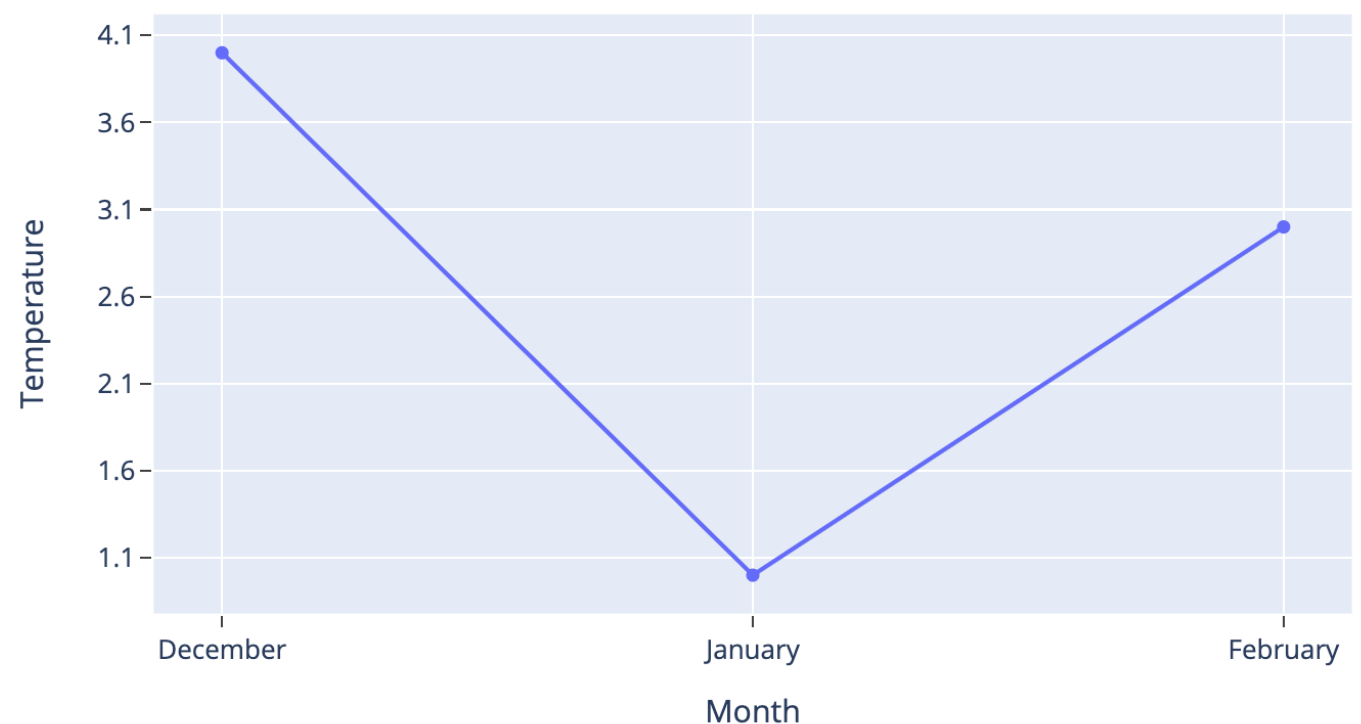
Placement, color, and design

```
fig = go.Figure(go.Scatter(  
    mode = "lines+markers",  
    y = [4, 1, 3],  
    x = ["December", "January", "February"]))
```

```
fig.update_xaxes(  
    title_text = "Month",  
    ticks='outside'  
)
```

```
fig.update_yaxes(  
    title_text = "Temperature",  
    ticks='outside',  
    nticks=20,  
    tick0=0.1,  
    dtick=0.5  
)
```

- Disabled by default. Use **'inside'** or **'outside'** to enable.
- **nticks** show an approximate number of ticks
- **tick0** and **dtick** set the initial value and the interval



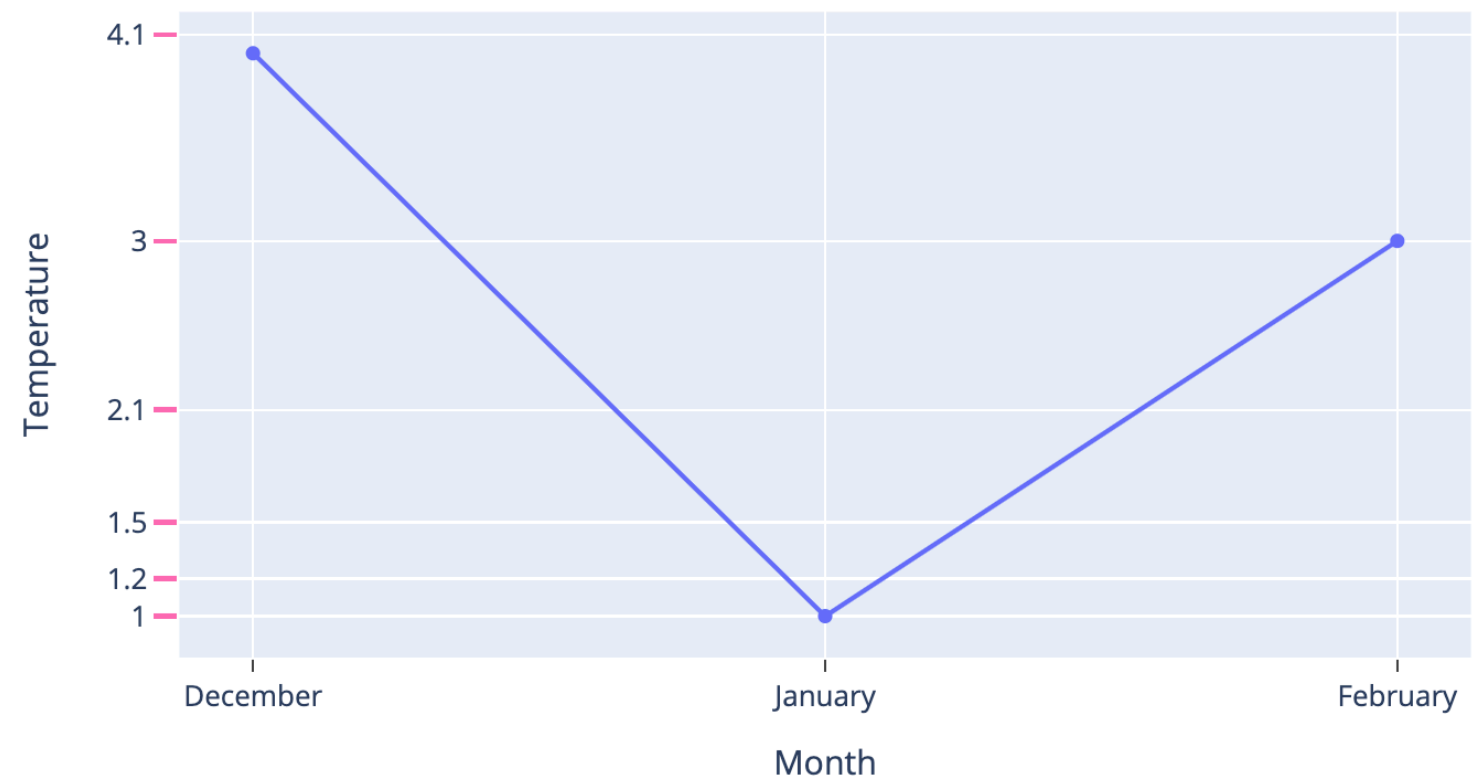
Ticks

Placement, color, and design

```
...  
fig.update_yaxes(  
    title_text = "Temperature",  
    ticks='outside',  
    tickvals=[1, 1.2, 1.5, 2.1, 3, 4.1],  
    tickwidth=2,  
    tickcolor='hotpink',  
    ticklen=10  
)  
...
```

Visualization crime! The step for the ticks should be natural

- **tickvals**: sets exact location of ticks
- **tickwidth**, **tickcolor**, **ticklen** corresponds to width, color and length of ticks, respectively.



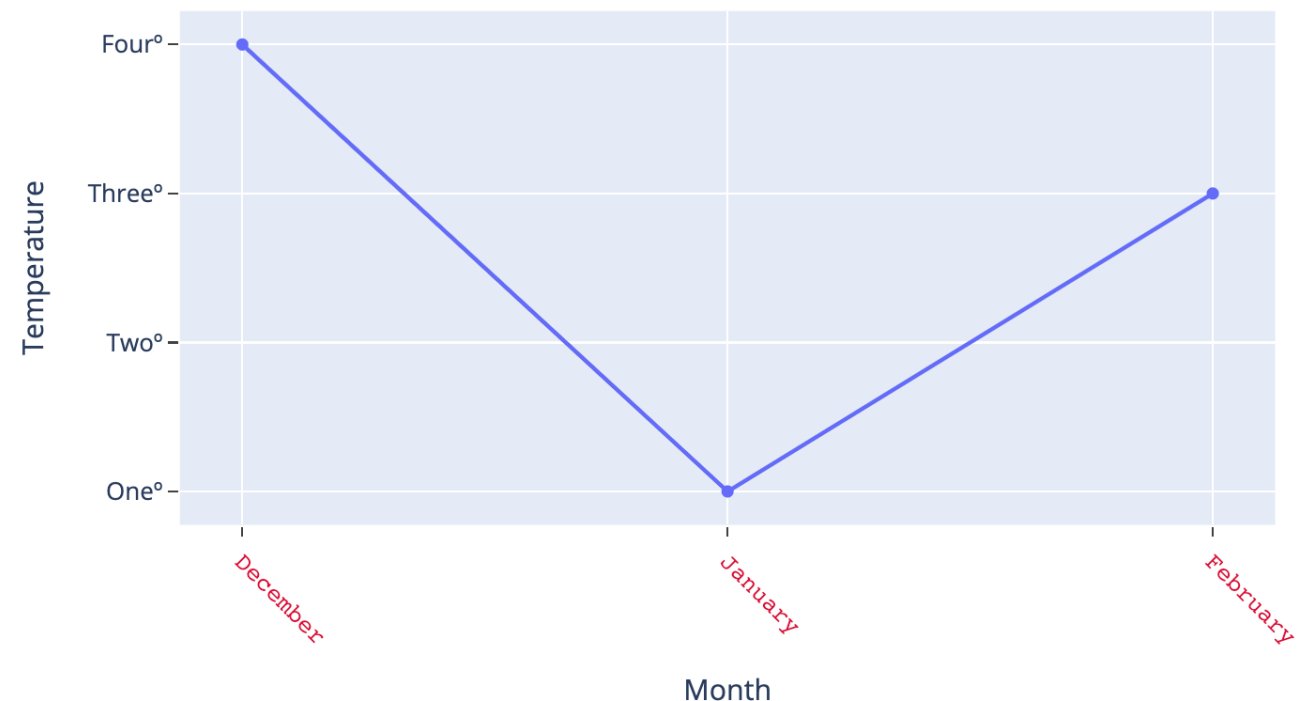
Labels

Font, rotation, and enumeration

```
fig.update_xaxes(  
    title_text = "Month",  
    ticks='outside',  
    tickangle=45,  
    tickfont=dict(family='Courier', size=12,  
color='crimson')  
)
```

```
fig.update_yaxes(  
    title_text = "Temperature",  
    ticks='outside',  
    tickvals=[1, 2, 3, 4],  
    ticktext=['One°', 'Two°', 'Three°',  
'Four°'],  
)
```

- **tickvals**: sets exact location of ticks;
- **ticktext**: contains the text that will be displayed at corresponding locations;
- **tickangle**: sets tick angle;
- **tickfont**: specifies different attributes for a font



Axis, grid, and zero lines

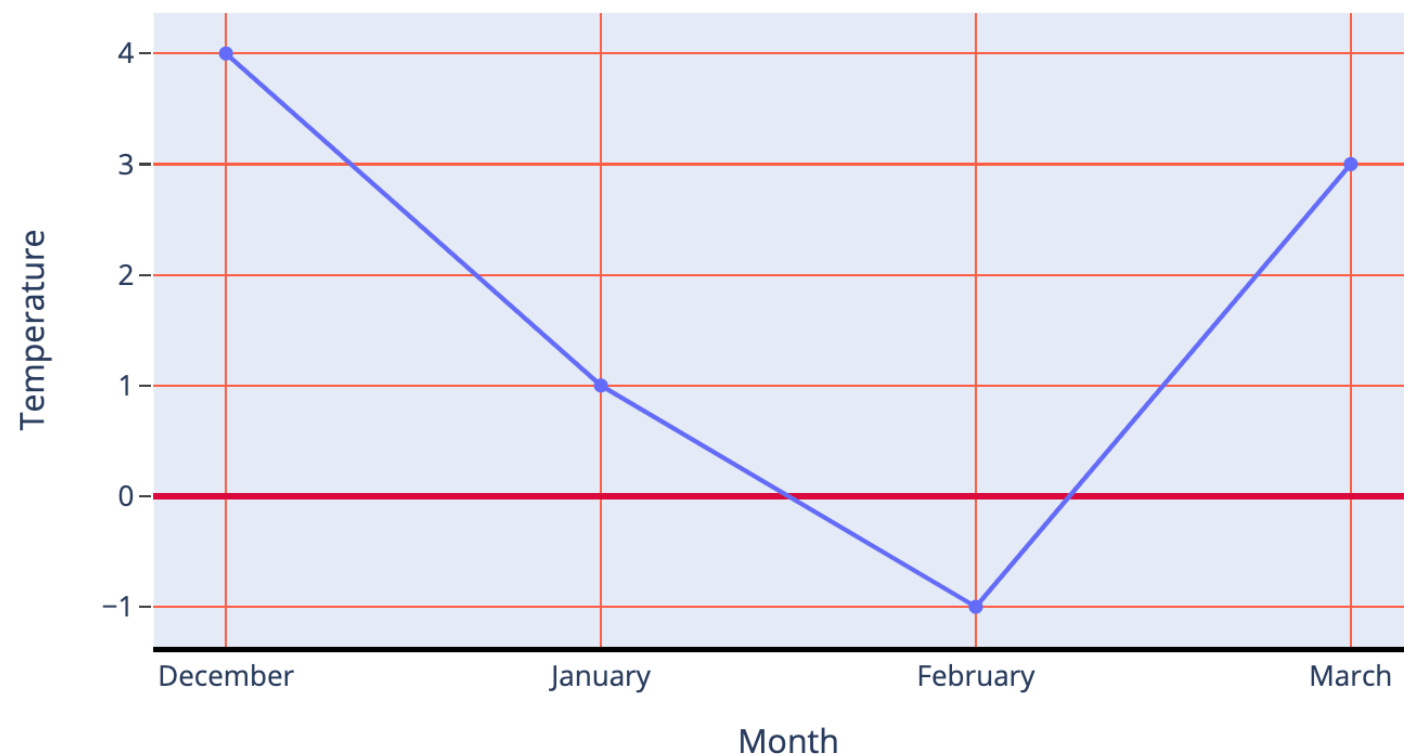
Setting color and width

```
fig = go.Figure(go.Scatter(  
    mode = "lines+markers",  
    y = [4, 1, -1, 3],  
    x = ["December", "January", "February",  
"March"])))
```

```
fig.update_xaxes(  
    title_text = "Month",  
    showline=True,  
    linewidth=2,  
    linecolor='black',  
    showgrid=True,  
    gridwidth=1,  
    gridcolor='tomato'  
)
```

```
fig.update_yaxes(  
    title_text = "Temperature",  
    ticks='outside',  
    showgrid=True,  
    gridwidth=1,  
    gridcolor='tomato',  
    zeroline=True,  
    zerolinewidth=3,  
    zerolinecolor='crimson'  
)
```

- Line = axis line. So, line on X and Y corresponds to X-axis and Y-axis.
- Zeroline = zero on the axis. So, zero line on X and Y function corresponds to Y-axis and X-axis (note the swap).
- Gridline = grid lines. Applies the same rule as for zero lines.



Ranges and domains

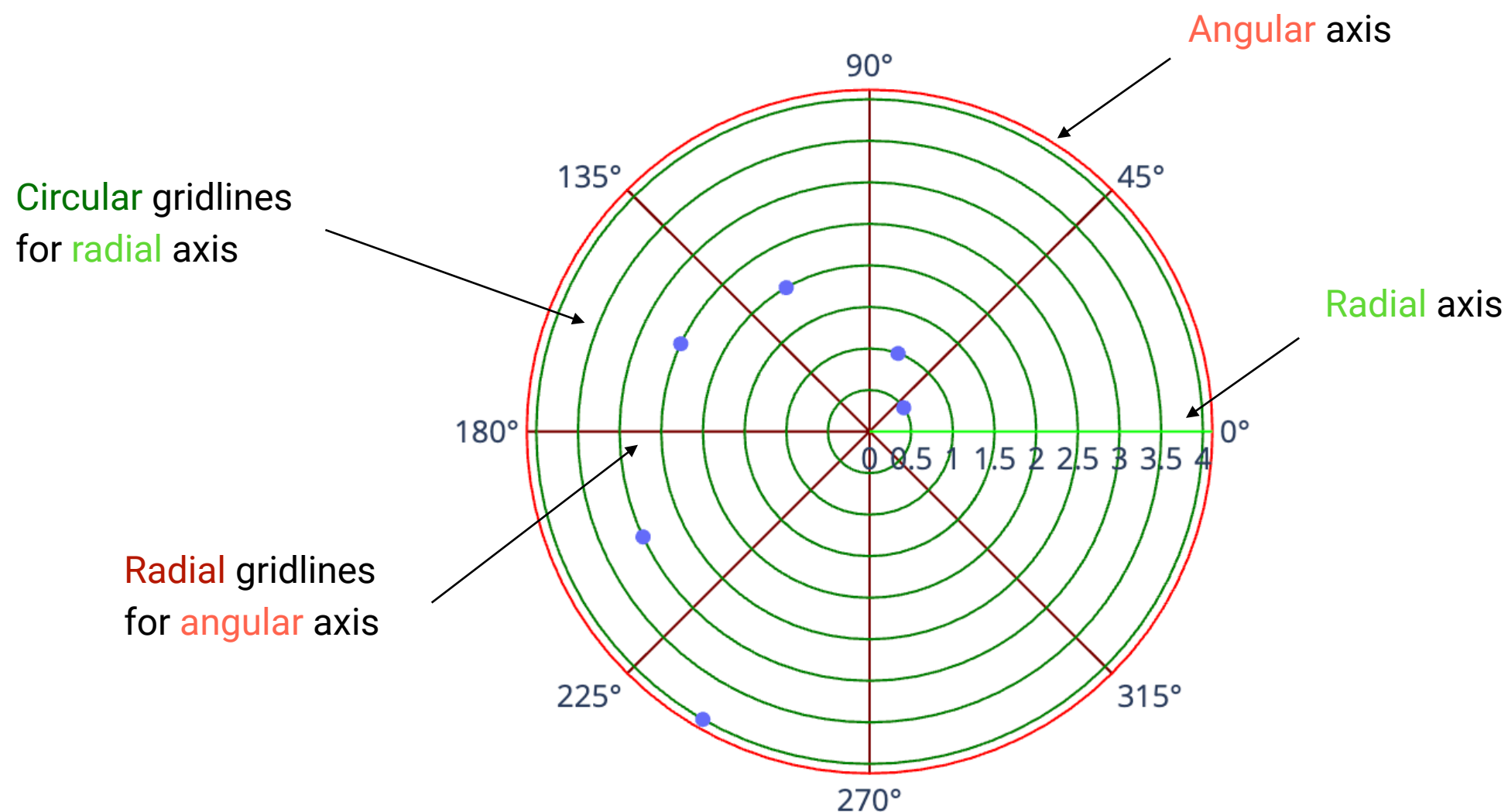
- **range**: manually sets the range of axis (e.g., `range=[-2, 5]`)
- **domain**: manually sets the domain values (e.g., `domain=[0.1, 0.9]`).
Default is `[0, 1]`, which corresponds to the entire area of the graphic.

Agenda

1. 2D Cartesian axis
- 2. Polar axis**
3. Color “axis”
4. Title and Legends

Polar axis

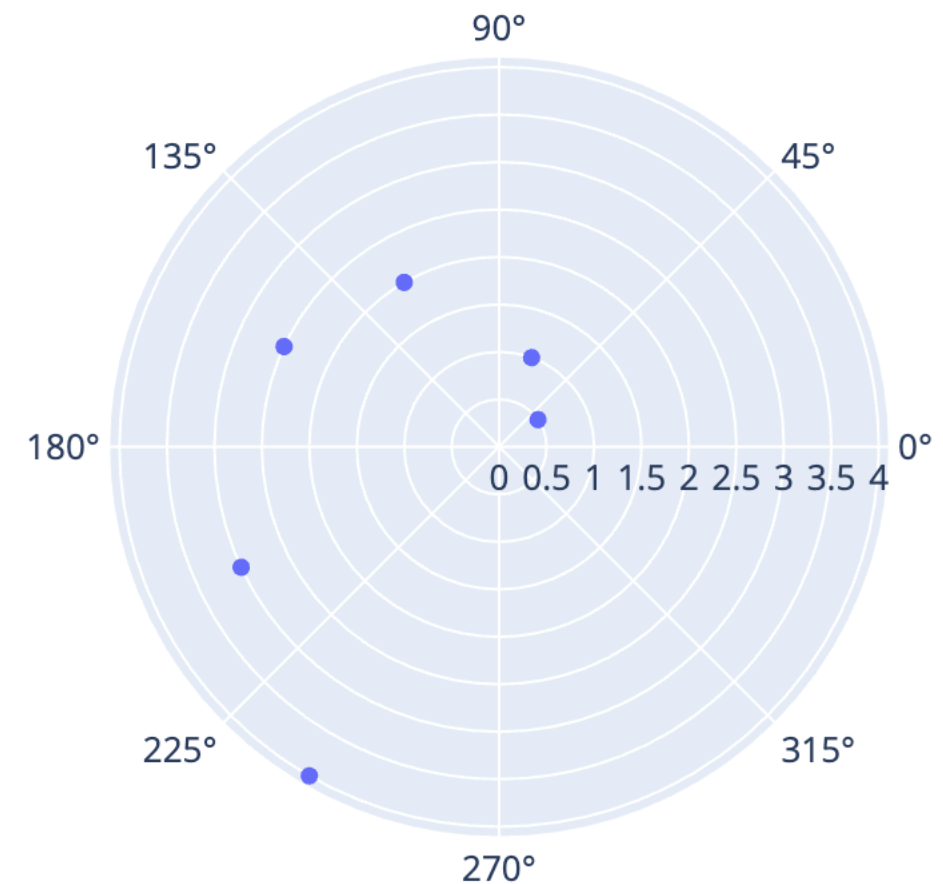
- Polar axis enable a new range of graphics that are constructed within polar coordinate system.
- A polar graph is a circular graph on which data are displayed in terms of values and angles.



Polar chart

```
data = go.Scatterpolar(  
    r = [0.5, 1, 2, 2.5, 3, 4],  
    theta = [35, 70, 120, 155, 205, 240],  
    mode = 'markers',  
)
```

```
fig = go.Figure(data=trace)  
fig.update_layout(showlegend=False)
```

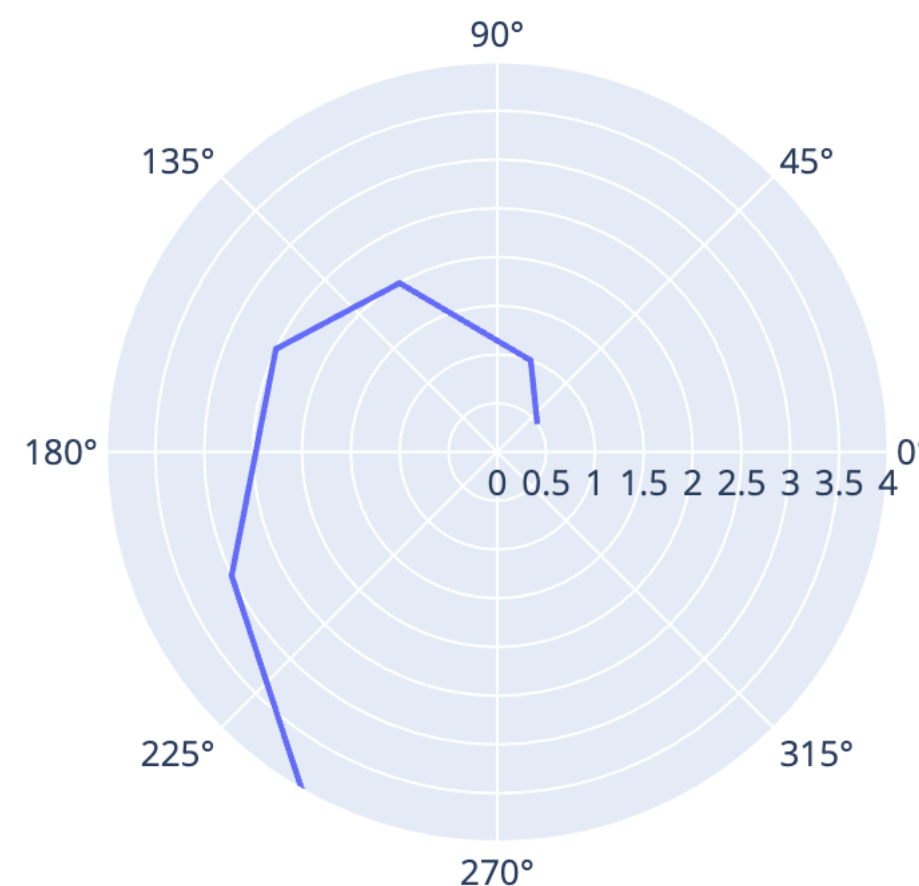


- Notice the use of “polar”
- **r** sets the **radial** coordinate
- **theta** sets the **angular** coordinate

Polar Line chart

```
data = go.Scatterpolar(  
    r = [0.5, 1, 2, 2.5, 3, 4],  
    theta = [35, 70, 120, 155, 205, 240],  
    mode = 'lines',  
)
```

```
fig = go.Figure(data=trace)  
fig.update_layout(showlegend=False)
```

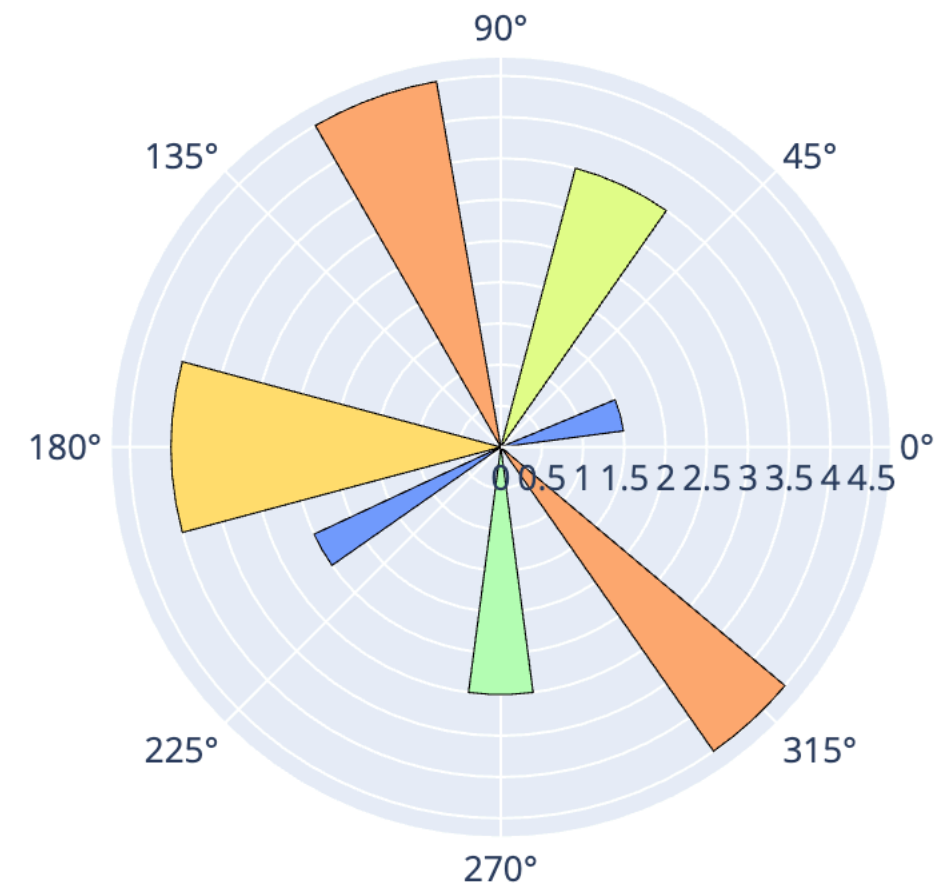


- Notice the use of "polar"
- **r** sets the **radial** coordinate
- **theta** sets the **angular** coordinate

Polar bar chart

Wind rose

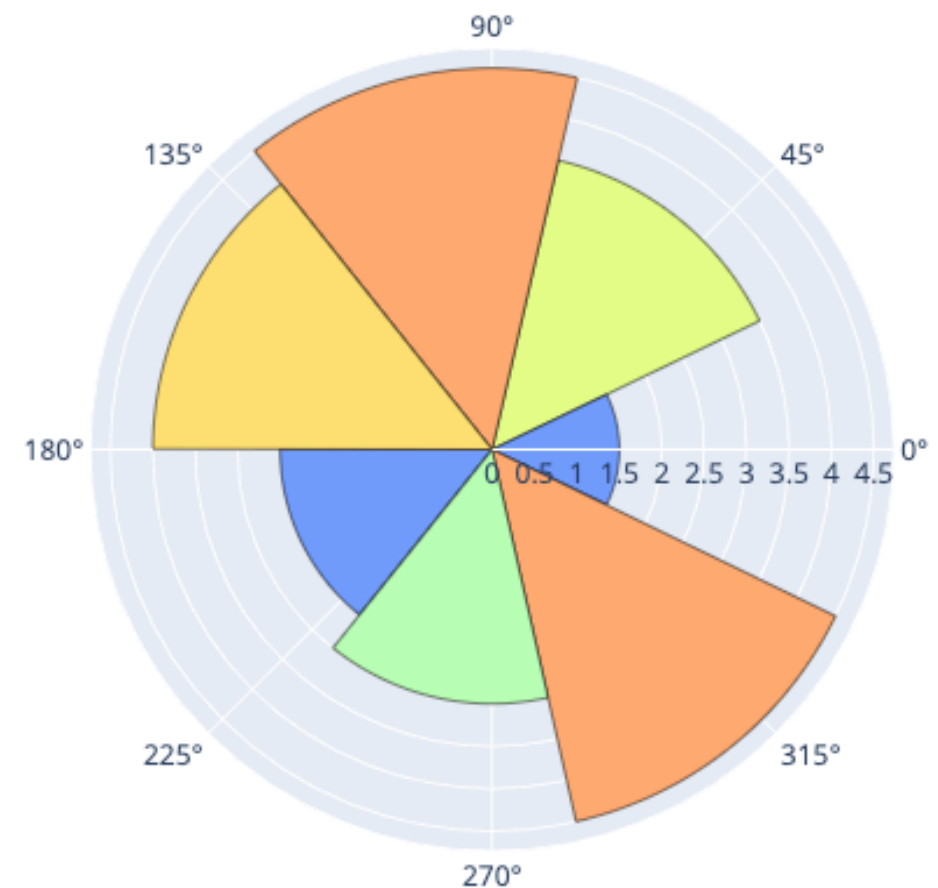
```
trace = go.Barpolar(  
    r=[3.5, 1.5, 2.5, 4.5, 4.5, 4, 3],  
    theta=[65, 15, 210, 110, 312.5, 180, 270],  
    width=[20, 15, 10, 20, 15, 30, 15, ],  
    marker_color=["#E4FF87", '#709BFF', '#709BFF', '#FFAA70', '#FFAA70', '#FFDF70',  
    '#B6FFB4'],  
    marker_line_color="black"  
)
```



Polar bar chart

Wind rose

```
trace = go.Barpolar(  
    r=[3.5, 1.5, 2.5, 4.5, 4.5, 4, 3],  
    theta=[51.4, 0, 205.6, 308.4, 102.8, 154.2, 257],  
    width=51.4,  
    marker_color=["#E4FF87", '#709BFF', '#709BFF', '#FFAA70', '#FFAA70', '#FFDF70',  
    '#B6FFB4'],  
    marker_line_color="black"  
)
```

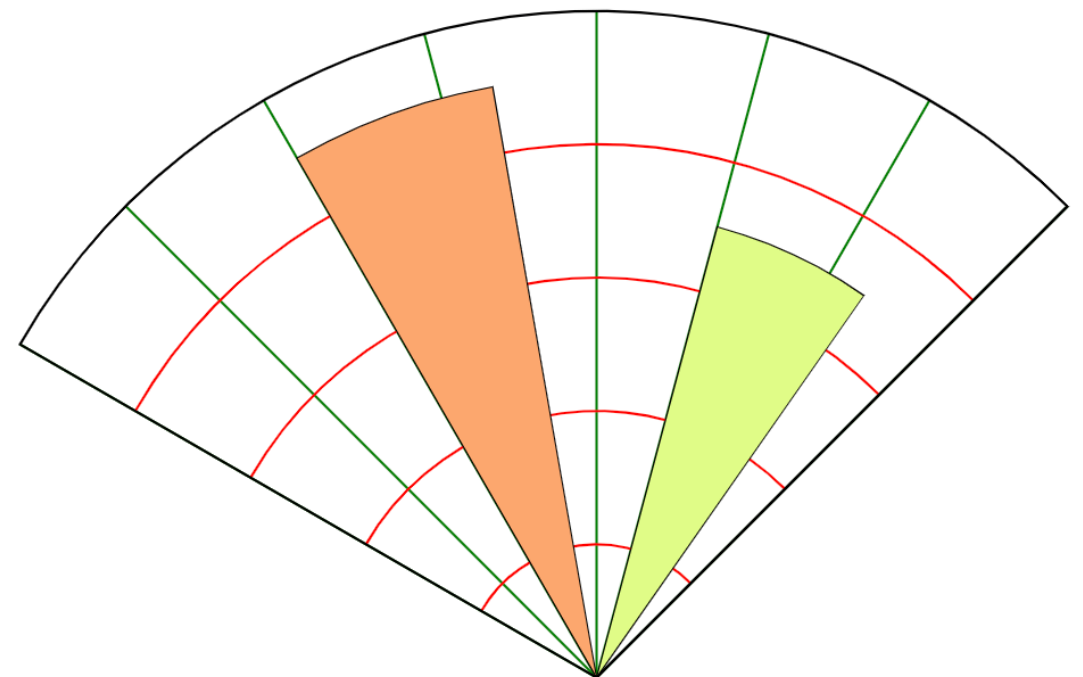
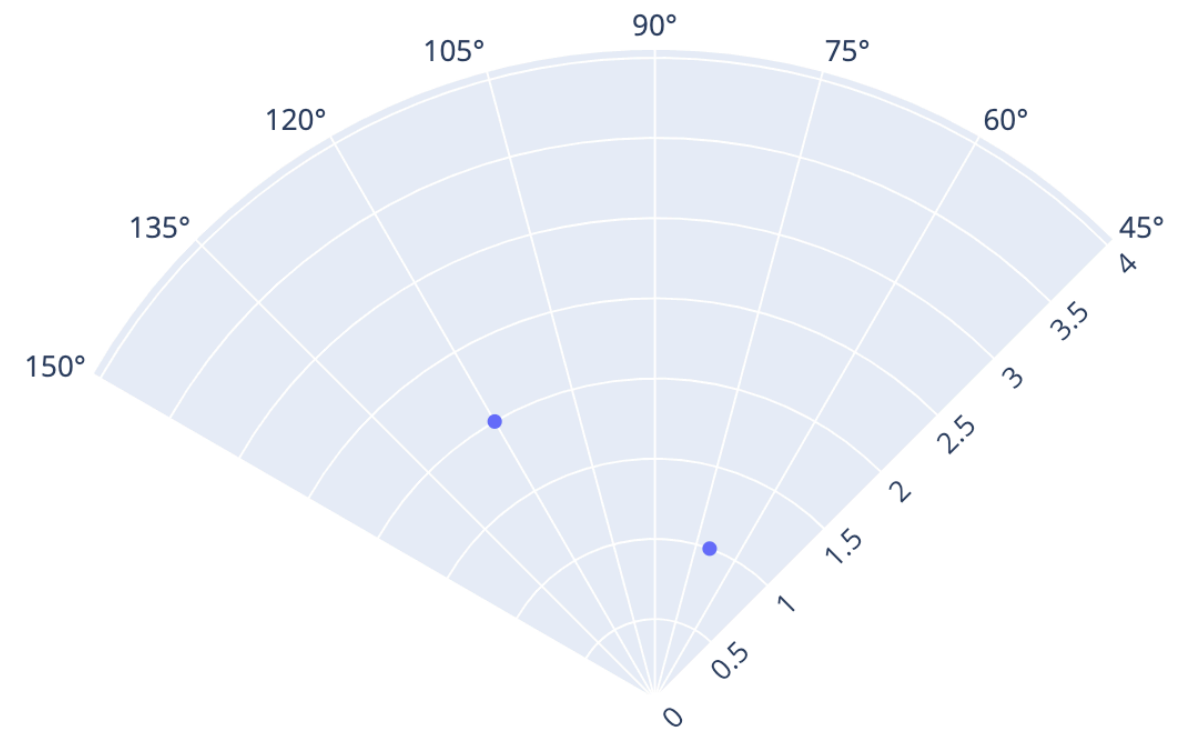


Polar chart sector

Wind rose

```
fig.update_polars(sector=[45, 150])
```

- **sector** sets the span of the polar chart in degrees in the counterclockwise direction.
- To set direction manually use **direction** attribute (`fig.update_polars(angularaxis_direction=<VALUE>)`), which can be "counterclockwise" or "clockwise"

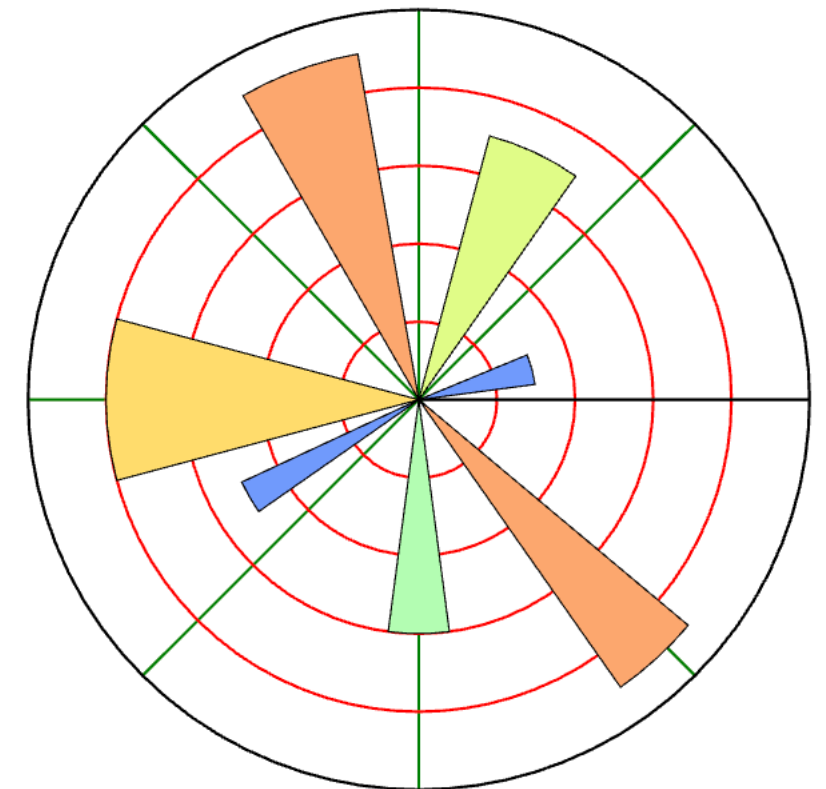


Polar axis

Design

```
fig.layout.polar.bgcolor='white'  
fig.layout.polar.radialaxis = dict(  
    range=[0, 5], showticklabels=False, ticks='',  
    linecolor = 'black',  
    gridcolor = 'red'  
)  
fig.layout.polar.angularaxis = dict(  
    showticklabels=False, ticks='',  
    linecolor = 'black',  
    gridcolor = 'green'  
)
```

- layout.Polar or fig.update_polars()
- radialaxis and angularaxis for styling radial and angular axes, respectively.

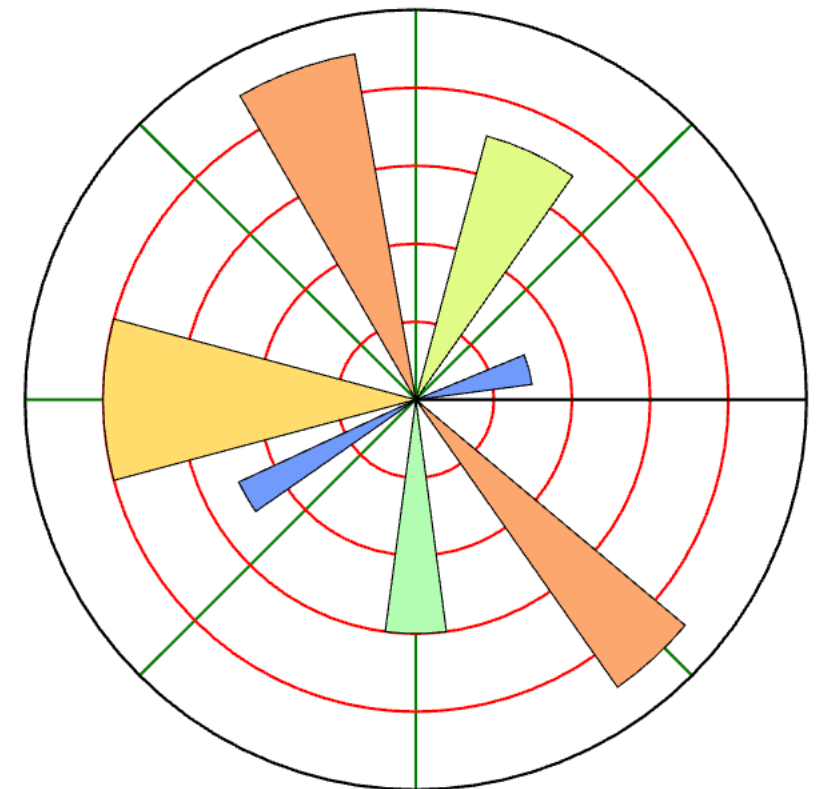


Polar axis

Design

```
fig.update_polars(  
    bgcolor='white',  
    radialaxis = dict(  
        range=[0, 5], showticklabels=False, ticks='',  
        linecolor = 'black',  
        gridcolor = 'red'  
    ),  
    angularaxis = dict(  
        showticklabels=False, ticks='',  
        linecolor = 'black',  
        gridcolor = 'green'  
    )  
)
```

- layout.Polar or fig.update_polars()
- radialaxis and angularaxis for styling radial and angular axes, respectively.

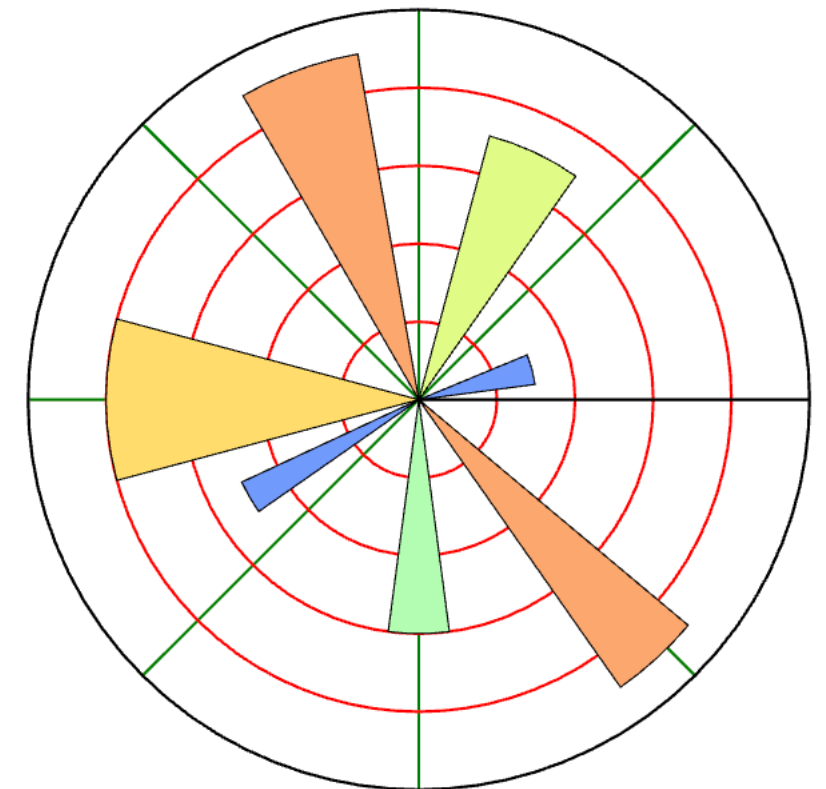


Polar axis

Design

```
fig.update_layout(  
    polar = dict(  
        bgcolor='white',  
        radialaxis = dict(  
            range=[0, 5], showticklabels=False, ticks='',  
            linecolor = 'black',  
            gridcolor = 'red'  
        ),  
        angularaxis = dict(  
            showticklabels=False, ticks='',  
            linecolor = 'black',  
            gridcolor = 'green'  
        )  
    )  
)
```

- layout.Polar or fig.update_polars()
- radialaxis and angularaxis for styling radial and angular axes, respectively.



Agenda

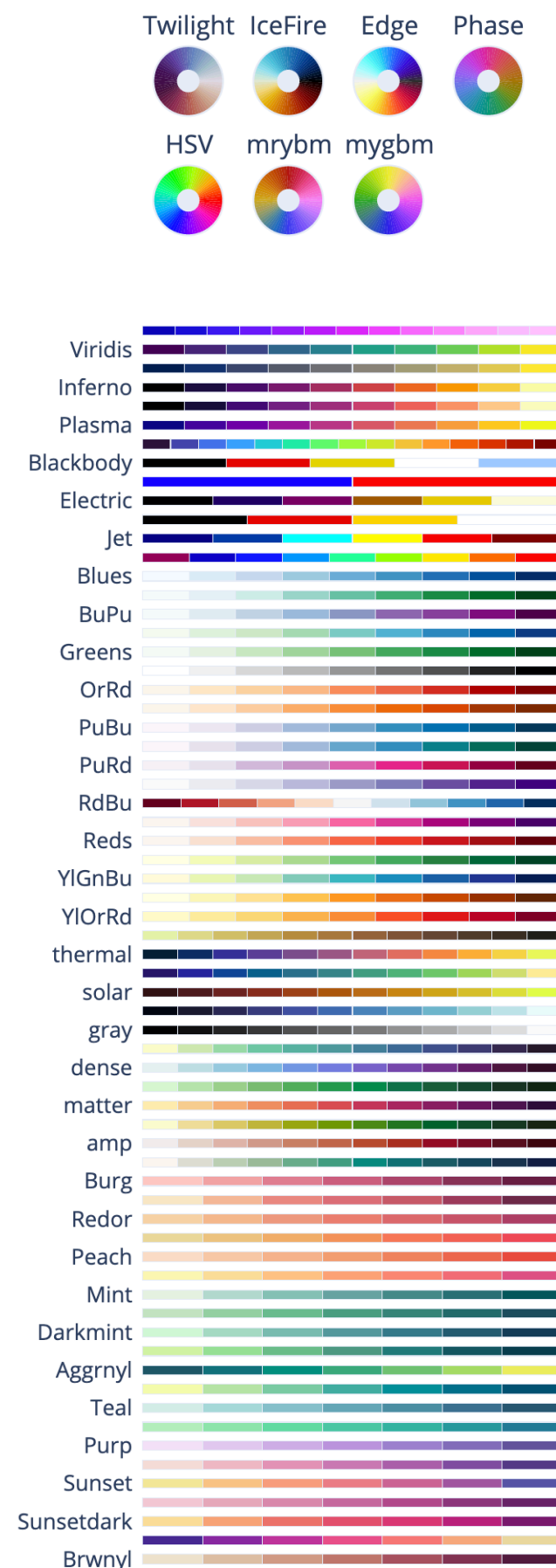
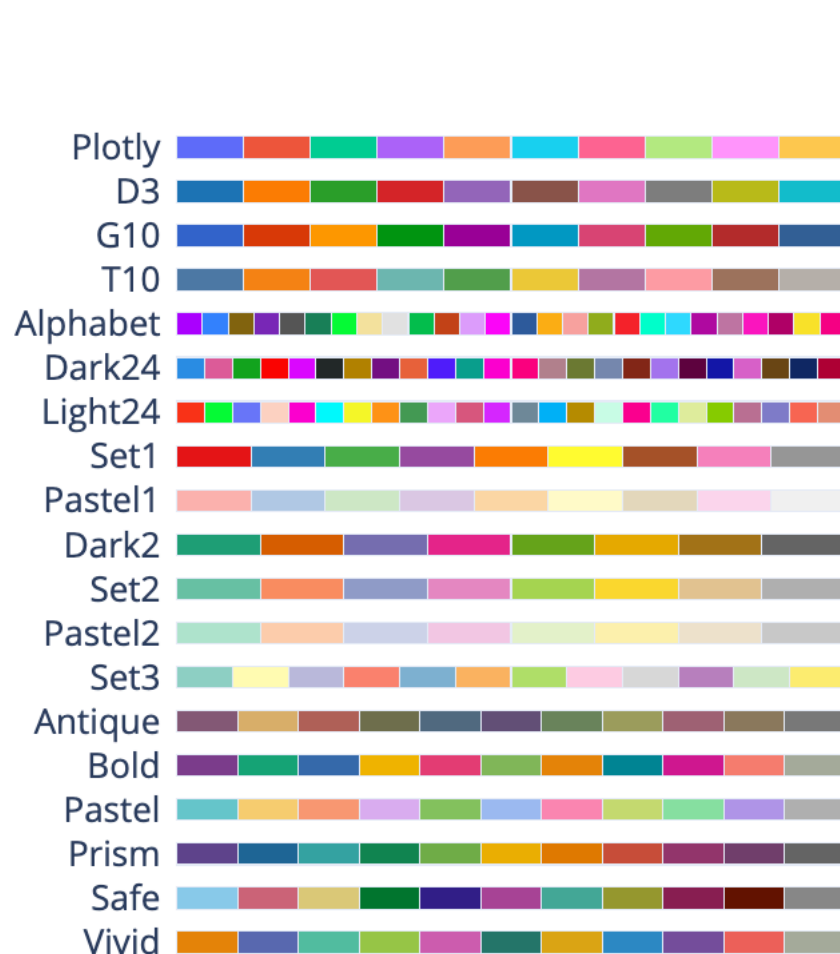
1. 2D Cartesian axis
2. Polar axis
- 3. Color “axis”**
4. Title and Legends

Color

Continuous vs discrete

- Continuous color schemes can be **sequential** | **diverging** | **cyclic**

→
↔
↻
- Discrete** (a.k.a qualitative) color schemes use a predefined number of colors



Continuous color scales

Fundamental concepts

- Scales
- Ranges
- Bars
- Axes

Continuous color scales

Fundamental concepts

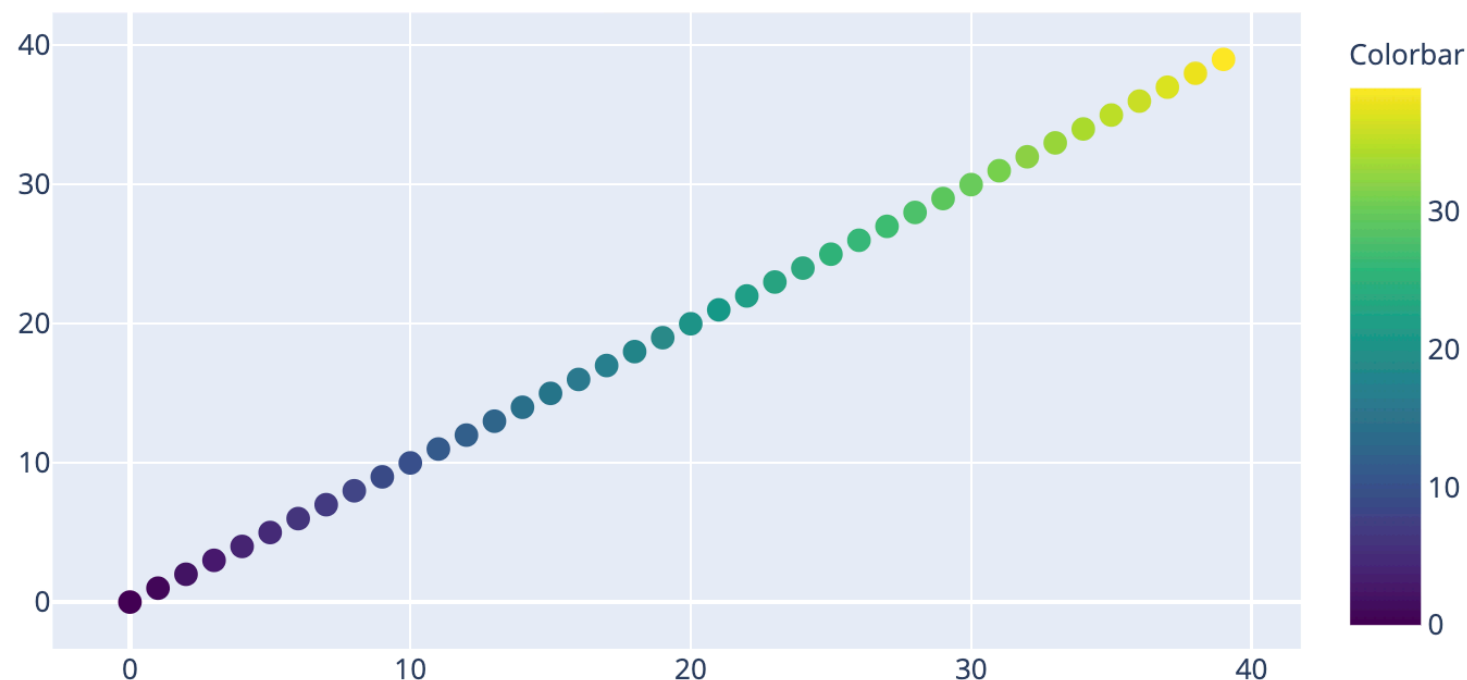
- Color **scales** represent a mapping between the range $[0, 1]$ and some color domain (e.g., red to green)
- Color **scales** can be explicitly specified using the **colorscale** argument in various graph_objects (e.g., **layout.coloraxis**, **marker.colorscale**)
- Color **ranges** represent the **minimum** to **maximum** range of data, which will be mapped to the $[0, 1]$ of the color scale.
- Color **ranges** can be specified using `cmin/cmidx/cmax` or `zmin/zmid/zmax` attributed of various graph_objects (e.g., `layout.coloraxis.cmin`, `marker.cmin`)
- Color **bars** are legends, which represent the color ranges and can be configured globally with **layout.coloraxis.colorbar** or locally with **marker.colorbar**
- Color **axis** connect color **scales**, color **ranges** and color **bars** to a trace's data

Continuous color scales

Scatter plot example

```
values = list(range(40))

fig.add_trace(go.Scatter(
    x=values,
    y=values,
    marker=dict(
        size=10,
        cmap=39,
        cmin=0,
        color=values,
        colorbar=dict(
            title="Colorbar"
        ),
        colorscale="Viridis"
    ),
    mode="markers"))
```

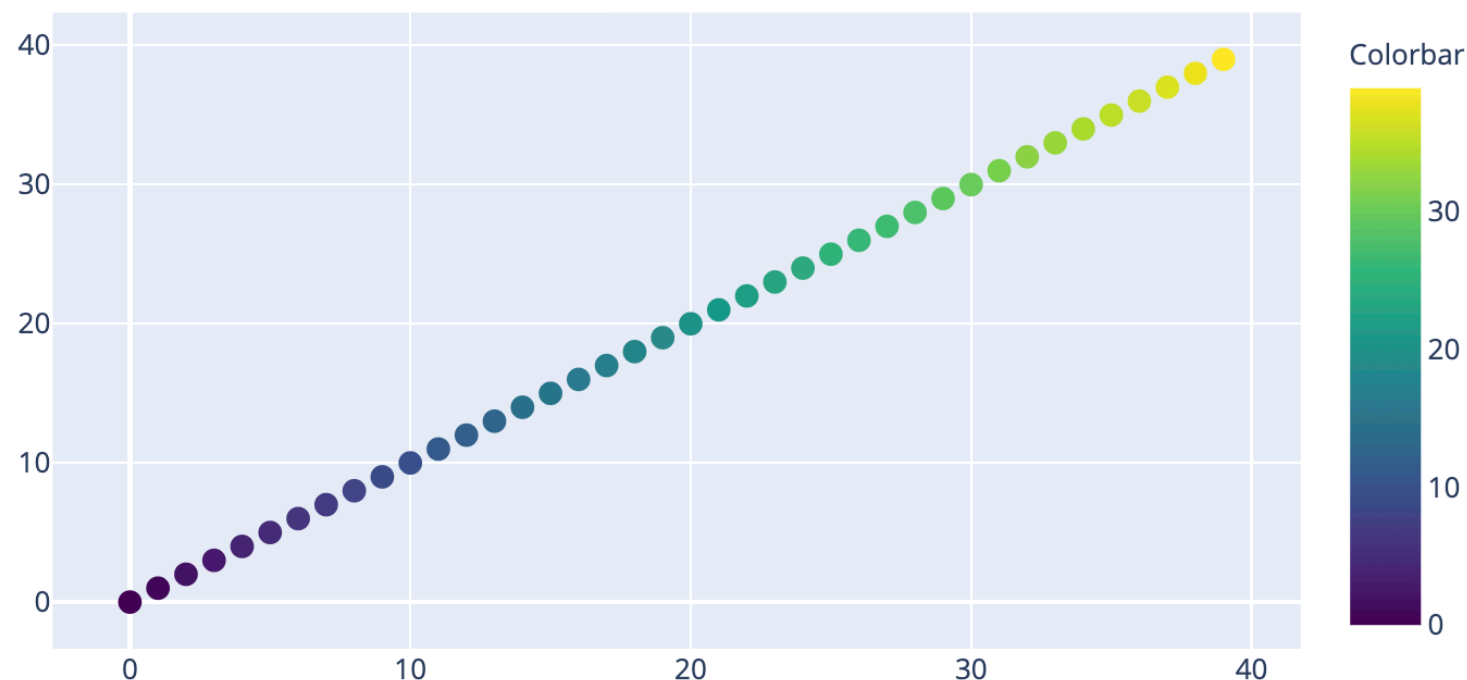


Continuous color scales

Scatter plot example

```
values = list(range(40))

fig.add_trace(go.Scatter(
    x=values,
    y=values,
    marker=dict(
        size=10,
        cmap=39,
        cmin=0,
        color=values,
        colorbar=dict(
            title="Colorbar"
        ),
        colorscale=plotly.colors.sequential.Viridis
    ),
    mode="markers"))
```

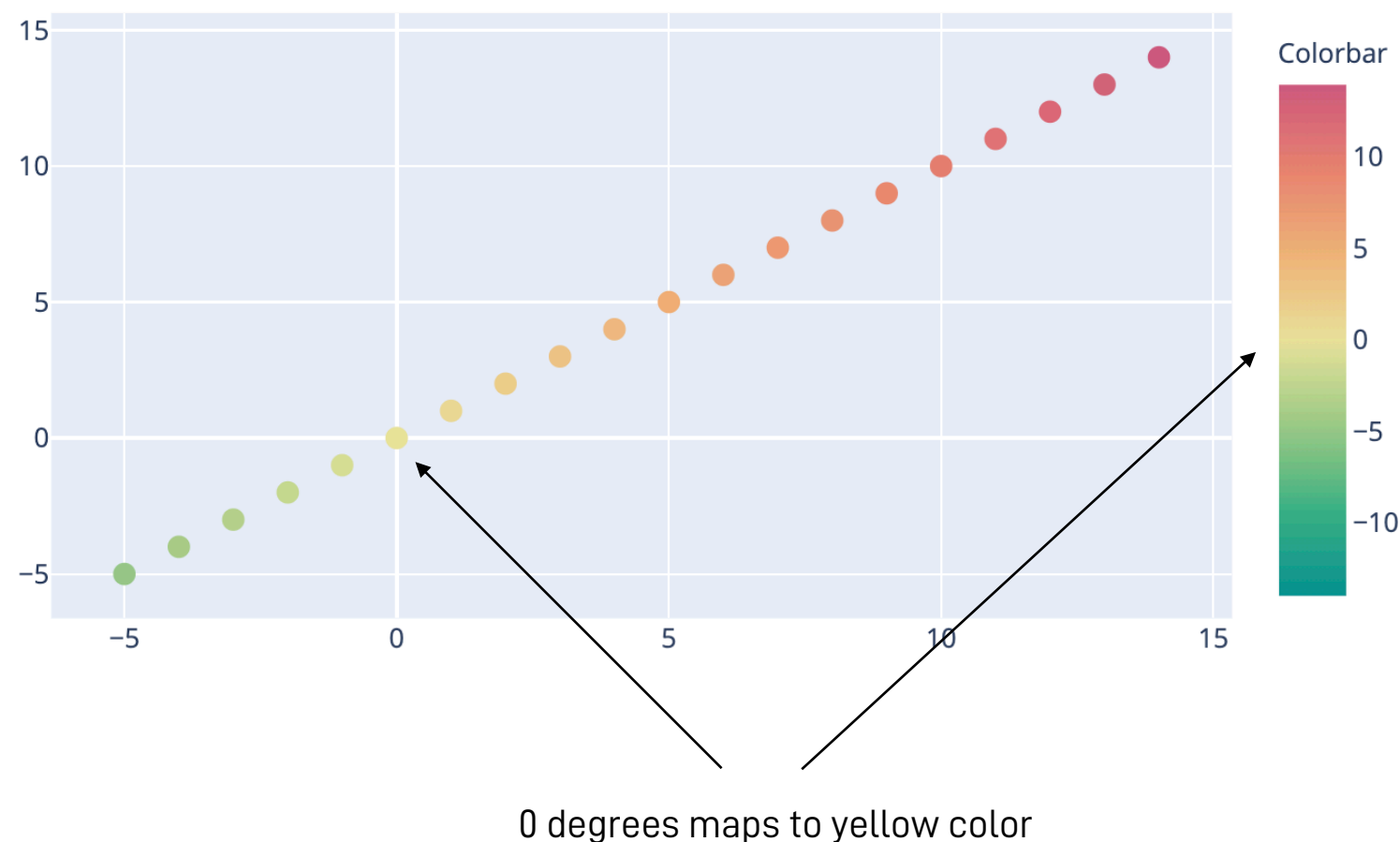


Continuous color scales

Diverging color schemes example

```
values = list(range(-5, 15))

fig.add_trace(go.Scatter(
    x=values,
    y=values,
    marker=dict(
        size=10,
        cauto=True,
        cmid=0,
        color=values,
        colorbar=dict(
            title="Colorbar"
        ),
        colorscale='Temps'
    ),
    mode="markers"))
```

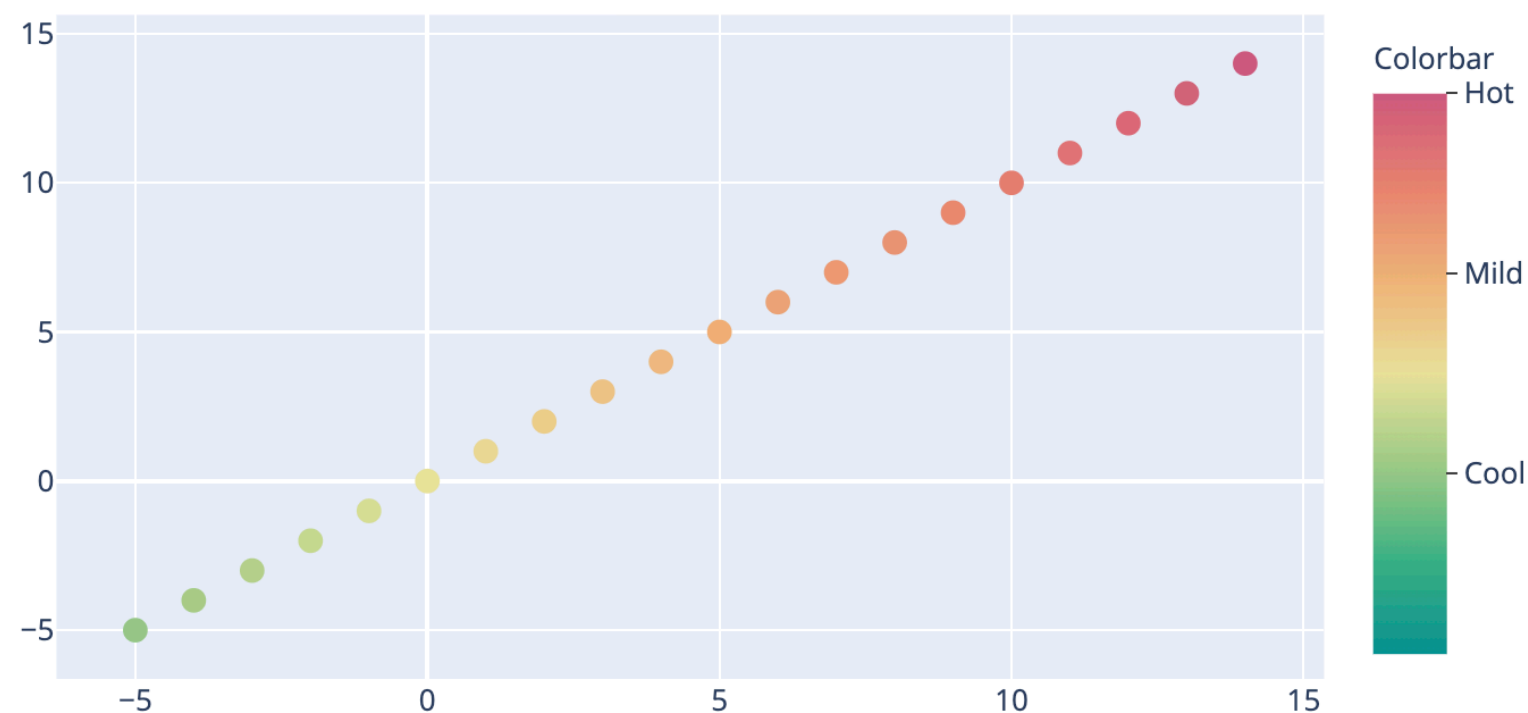


- Cmid sets the mid-point for diverging color scheme, scaling cmin and cmax to be equidistant.
- Only works if cauto is True

Continuous color scales

Color bar

```
fig.add_trace(go.Scatter(  
    x=values,  
    y=values,  
    marker=dict(  
        size=10,  
        cauto=True,  
        cmid=0,  
        color=values,  
        colorbar=dict(  
            title="Colorbar",  
            titleside="top",  
            tickmode="array",  
            tickvals=[-5, 5, 14],  
            ticktext=["Cool", "Mild", "Hot"],  
            ticks="outside"  
        ),  
        colorscale='Temps'  
    ),  
    mode="markers"))
```



Agenda

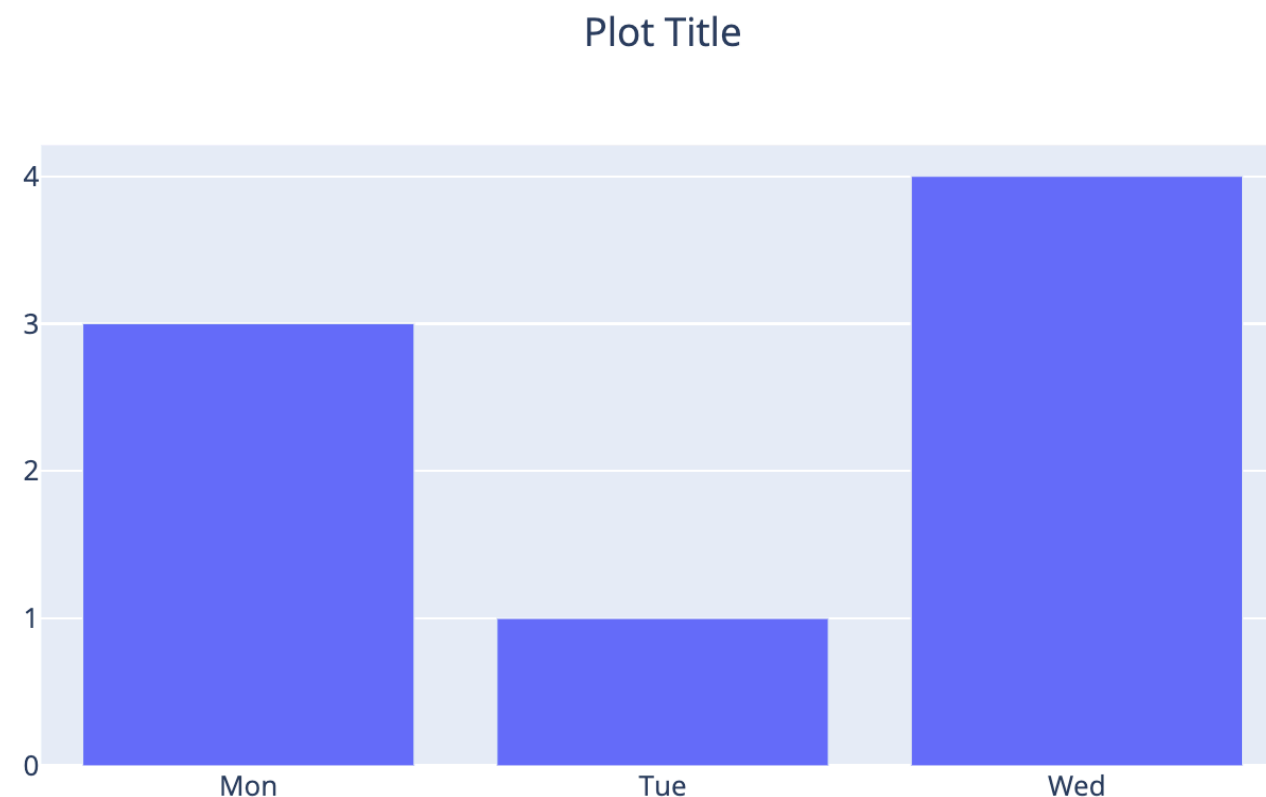
1. 2D Cartesian axis
2. Polar axis
3. Color “axis”
- 4. Title and Legends**

Setting title

layout.Title attribute, such as **text**, **x**, **y**, **xanchor** and **yanchor**

```
fig = go.Figure(go.Bar(
    y=[3, 1, 4],
    x=["Mon", "Tue", "Wed"]))

fig.update_layout(
    title= dict(
        text = "Plot Title",
        y = 0.9,
        x = 0.5,
        xanchor = 'center',
        yanchor = 'top'
    )
)
```



Legends

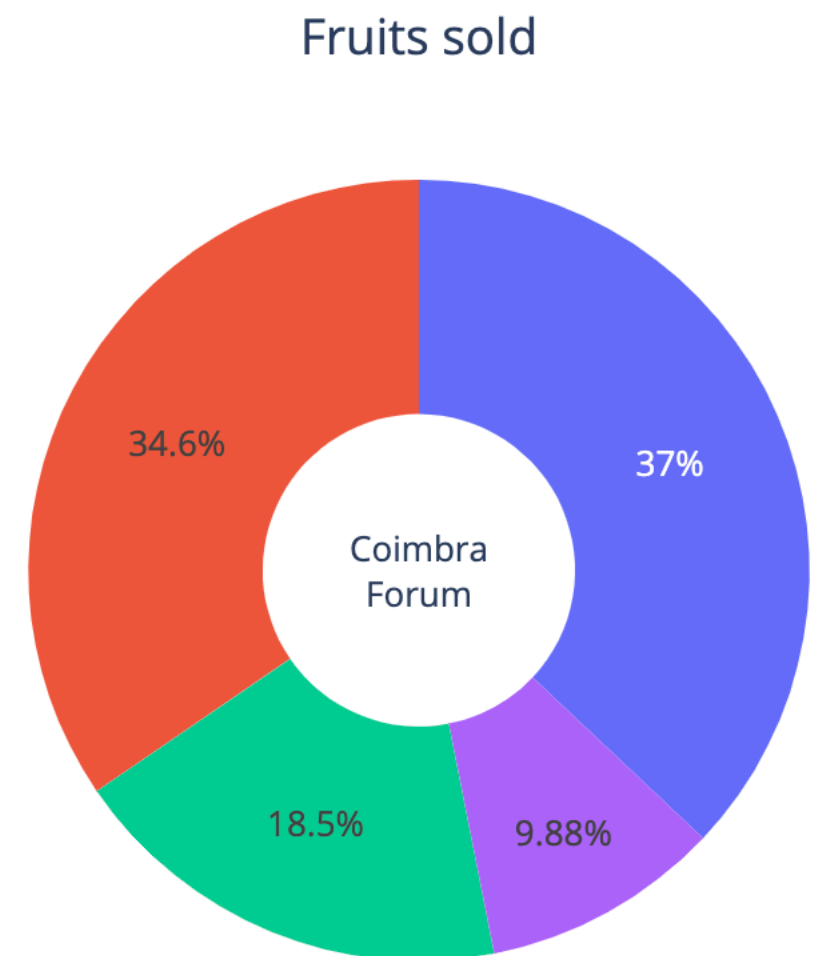
Setting positions

layout.Legend attribute, such as **title**, **x**, **y**, **xanchor** and **yanchor**

```
fig.update_layout(  
    legend = dict(  
        title = "Fruits",  
        yanchor="top",  
        y=0.99,  
        xanchor="left",  
        x=0.01  
    )  
)
```

Fruits

- Apples
- Oranges
- Bananas
- Pears

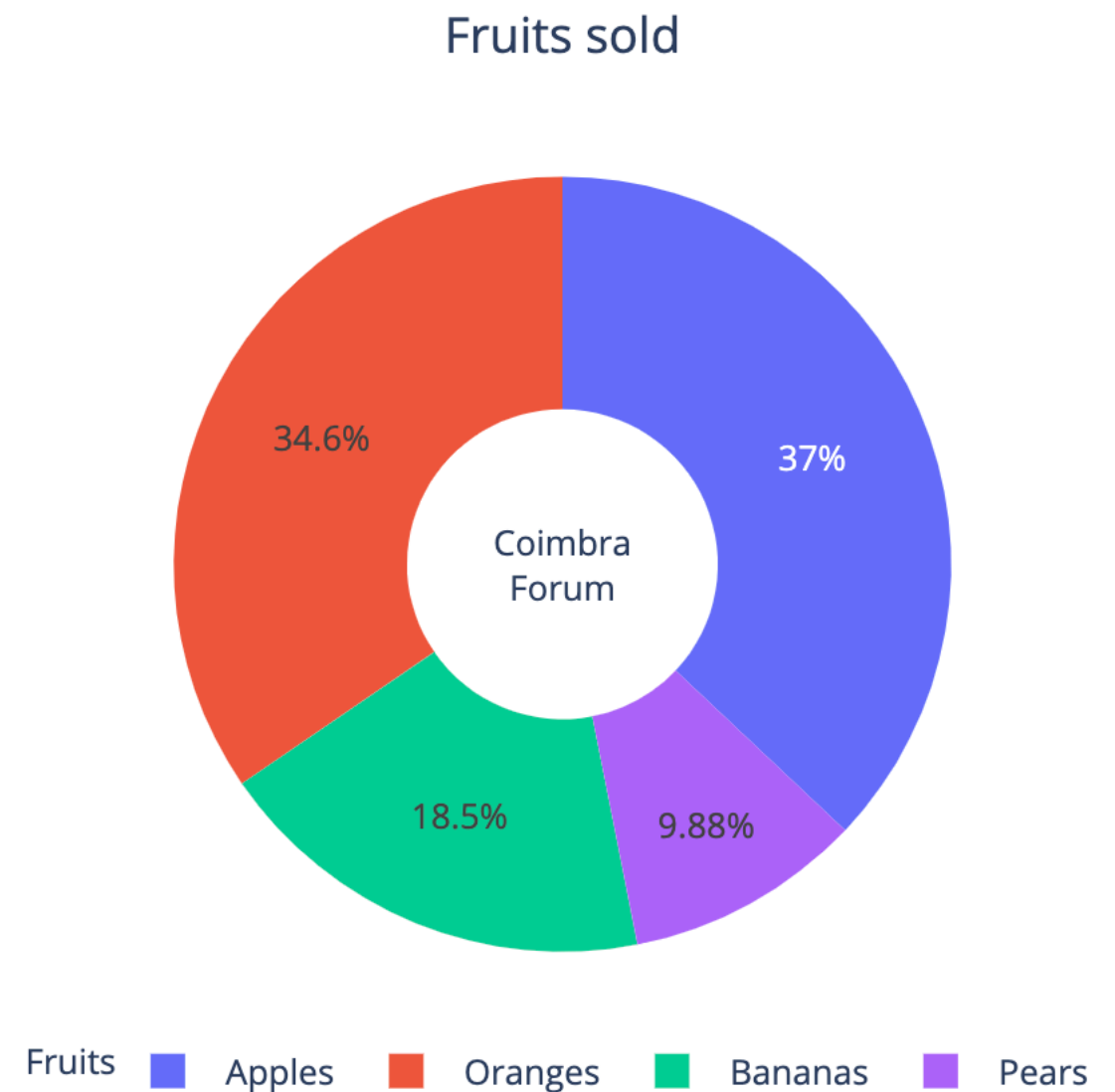


Legends

Horizontal

By setting the orientation attribute to 'h'

```
fig.update_layout(  
    legend = dict(  
        title = "Fruits",  
        yanchor="top",  
        y=-0.1,  
        xanchor="center",  
        x=0.5,  
        orientation='h'  
    )  
)
```



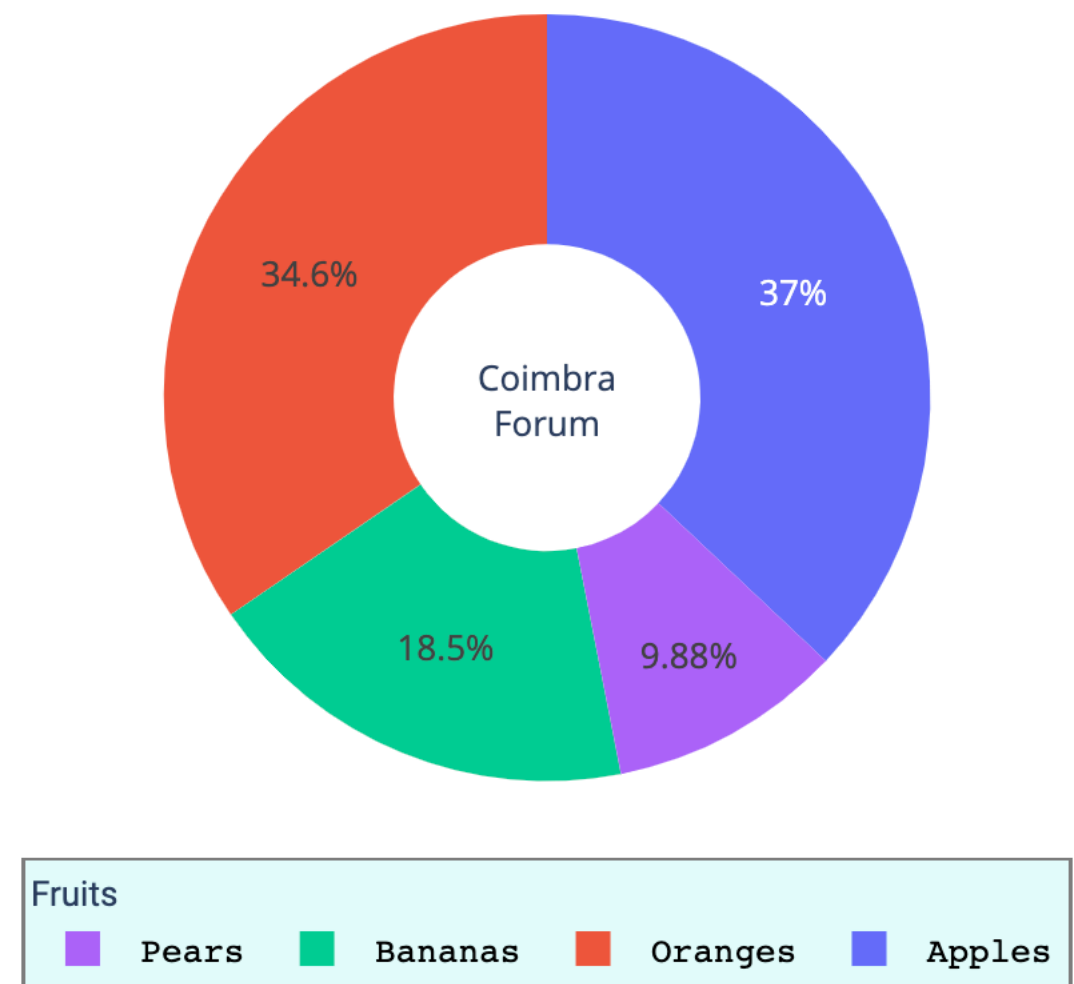
Legends

More attributes

Fonts, colors, trace order

```
fig.update_layout(  
    legend=dict(  
        x=0.5,  
        y=-0.1,  
        traceorder="reversed",  
        title_font_family="Roboto",  
        title_font_size = 12,  
        title_side = 'top',  
        font=dict(  
            family="Courier",  
            size=12,  
            color="black"  
        ),  
        bgcolor="lightCyan",  
        bordercolor="gray",  
        borderwidth=1  
    )  
)
```

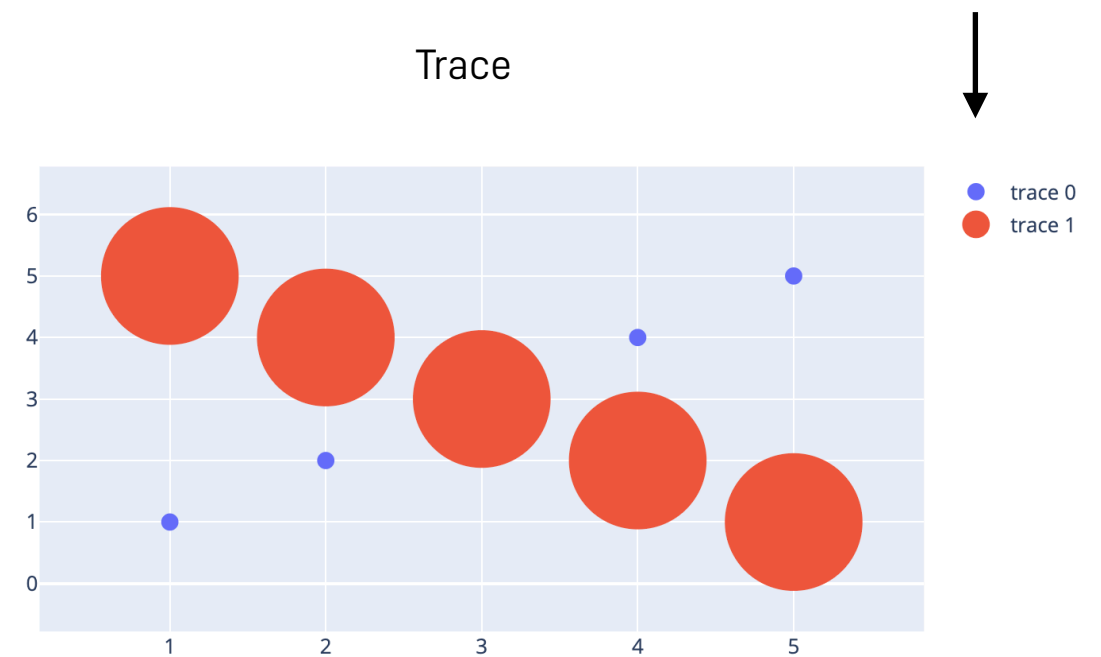
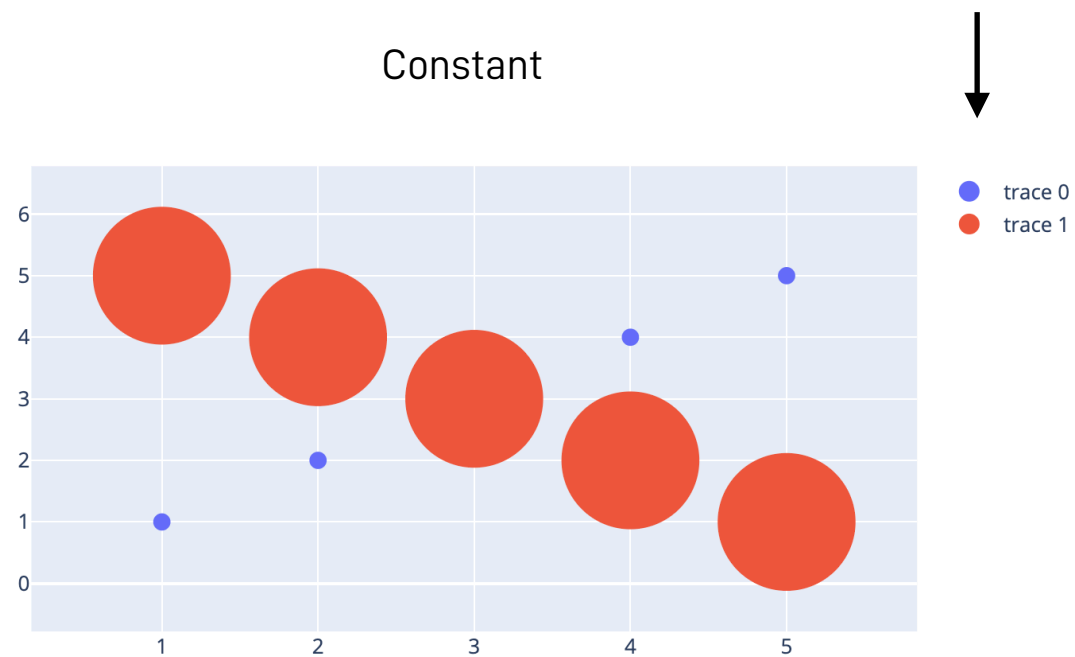
Fruits sold



Legends

Fixed symbol sizes

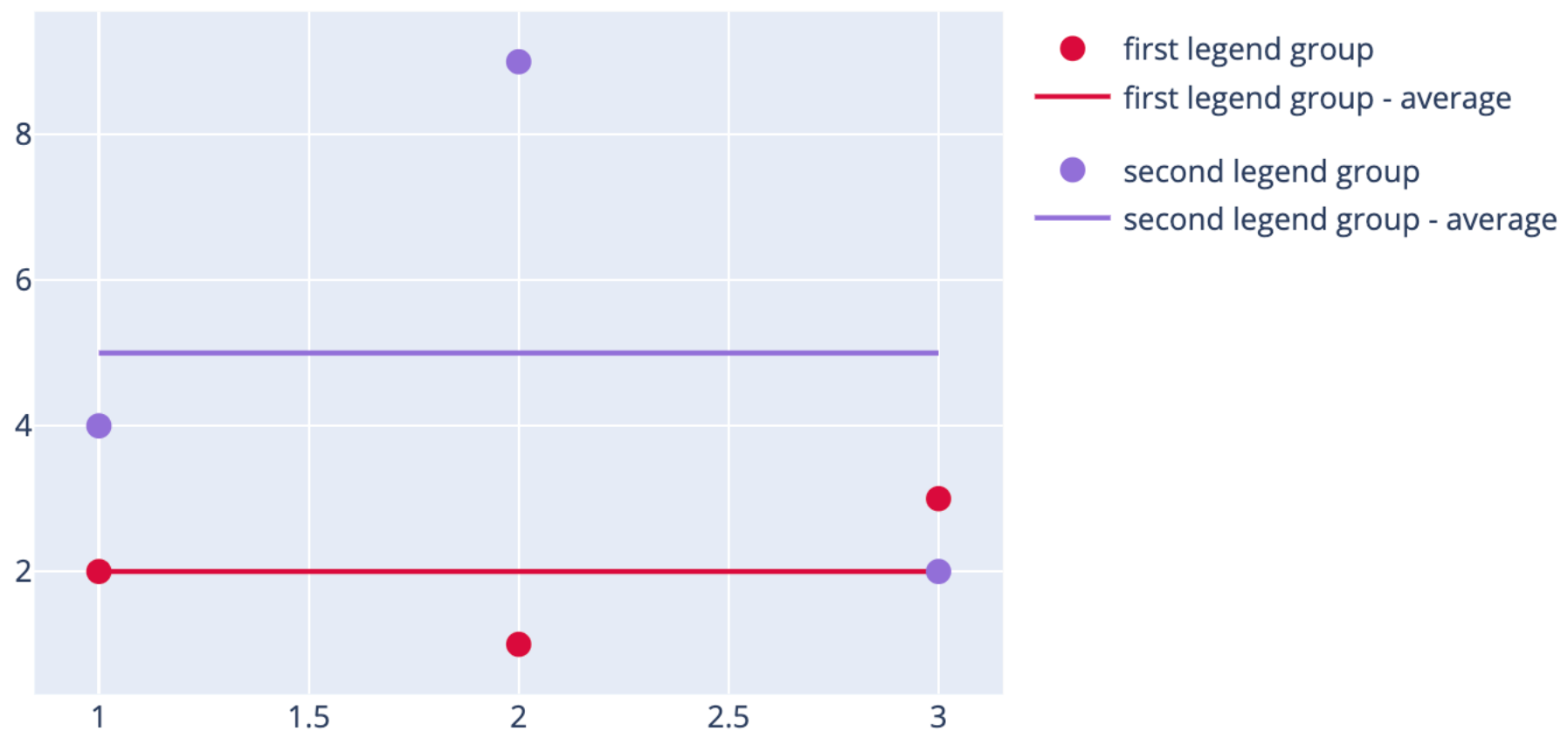
- `fig.update_layout(legend= {'itemsizing': 'constant'})`
- or 'trace'



Legends

Grouping

- Controlled by the **legendgroup** attribute, set per trace
- `go.Scatter(x=[1, 2, 3], y=[1, 2, 3], legendgroup="group1")`



References

- xaxis documentation: <https://plotly.com/python/reference/layout/xaxis/>
- Mode on axes: <https://plotly.com/python/axes/>
- Legend documentation: <https://plotly.com/python/reference/layout/#layout-legend>
- Title documentation: <https://plotly.com/python/reference/layout/>
- Scatter polar documentation: <https://plotly.com/python/reference/scatterpolar/>
- Bar polar documentation: <https://plotly.com/python/reference/barpolar/>
- Color axis: <https://plotly.com/python/reference/layout/coloraxis/>
- More on color: <https://plotly.com/python-api-reference/generated/plotly.colors.html#module-plotly.color>
- Polar axis: <https://plotly.com/python/reference/layout/polar/>